Build: Tic-Tac-Toe Game

Problem: Build a playable Tic-Tac-Toe game with win detection and reset.

What This Tests: 2D state management, game logic, derived state (winner), conditional rendering.

jsx

JSX Copy

```jsx
function TicTacToe() {
  const [board, setBoard] = useState(Array(9).fill(null));
  const [isXTurn, setIsXTurn] = useState(true);

  const winner = calculateWinner(board);
  const isDraw = !winner && board.every(cell => cell !== null);

  const handleClick = (index) => {
    if (board[index] || winner) return;
    const newBoard = [...board];
    newBoard[index] = isXTurn ? 'X' : 'O';
    setBoard(newBoard);
    setIsXTurn(!isXTurn);
  };

  const reset = () => {
    setBoard(Array(9).fill(null));
    setIsXTurn(true);
  };

  let status;
  if (winner) status = `Winner: ${winner}`;
  else if (isDraw) status = "It's a draw!";
  else status = `Next player: ${isXTurn ? 'X' : 'O'}`;
```

```
      <h2>{status}</h2>
      <div style={{ display: 'grid', gridTemplateColumns: 'repeat(3, 80px)
        {board.map((cell, i) => (
          <button
            key={i}
            onClick={() => handleClick(i)}
            style={{ width: 80, height: 80, fontSize: 28, fontWeight: 'bol
          >
            {cell}
          </button>
        ))}
      </div>
      <button onClick={reset} style={{ marginTop: 16 }}>Reset Game</button
    </div>
  );
}

function calculateWinner(board) {
  const lines = [
    [0,1,2], [3,4,5], [6,7,8],
    [0,3,6], [1,4,7], [2,5,8],
    [0,4,8], [2,4,6],
  ];
  for (const [a, b, c] of lines) {
    if (board[a] && board[a] === board[b] && board[a] === board[c]) {
      return board[a];
    }
  }
  return null;
}
```

Key Concepts: Flat array for 2D grid, derived state for winner and draw, preventing clicks on filled cells or after game ends, pure function for win detection, CSS Grid for layout.

QUICK CHECK — no marks, just for you.

1. In handleClick, the first line is a guard that returns early when board[index] is truthy or winner is truthy. What two situations does this guard exist to prevent?

Clicking outside the 3x3 grid bounds, and clicking before the board array has been initialised

- Overwriting a cell that already holds X or O, and allowing further moves once the game has been won
- Re-rendering the grid unnecessarily, and calling setState twice in the same event handler
- Letting the same player move twice in a row, and letting a move happen during a draw

Check answer

2. The board is modelled as Array(9).fill(null) rather than as three nested arrays of three cells. How does calculateWinner take advantage of this choice?

- It relies on Array.prototype.flat() to collapse the rows before comparing cells
- It declares each winning line as a triple of single indices into the one-dimensional array, so a line is checked by direct index lookup
- It uses the array length of 9 to compute row and column coordinates with division and modulo before comparing cells
- It maps over the board once per row and once per column, comparing the resulting sub-arrays for equality

Check answer

3. The component keeps only board and isXTurn in useState; winner and isDraw are plain constants recomputed on every render. Why is that the right design here?

- winner and isDraw are fully determined by board, so storing them in state would duplicate data that can go stale or fall out of sync
- React forbids storing booleans or strings in useState, so they must be recomputed each render

Putting them in useState would break the reset function, since Array(9).fill(null) cannot be passed to more than one setter

- [ ] Recomputing them on each render is required because useState values are not available during the render phase

Check answer

Previous
Build: Custom Dropdown / Select

Next
Build: Pagination Component

LEARN MORE

ARTICLES & DOCS

Tutorial: Tic-Tac-Toe (official)
react.dev

VIDEOS



Build tic-tac-toe in React
YouTube



Master React by building tic-tac-toe
YouTube



Tic-tac-toe — beginners tutorial
YouTube

Hand-picked references for this lesson.